

AI面接インタビューシステム

Day 7・Day 8 報告書

作成日: 2026年3月10日

リポジトリ: tianzhongyoushi178/AI_Interviewer

ブランチ: master

フレームワーク: Rails 6.1.7 / Ruby 3.1.6

目次

Day 7 — 自動リジェクト判定ロジック

1. 概要
2. DB変更（マイグレーション）
3. リジェクト判定サービス（RejectJudge）
4. リアルタイム判定フロー
5. 完了時最終判定
6. 既存コード改修
7. コードレビュー結果
8. 変更ファイル一覧

Day 8 — 音声AI面接の基盤実装

9. 概要
10. STTClient 強化（音声認識）
11. TTSCClient 強化（音声合成）
12. MediaProcessor 強化
13. AudioInterviewService（統合サービス）
14. コントローラー統合
15. 運用ツール（Rakeタスク）
16. コードレビュー結果
17. 変更ファイル一覧

付録: 全APIエンドポイント一覧 (Day 8時点)

Day 7 — 自動リジェクト判定ロジック

1. 概要

面接中のリアルタイムリジェクト判定と、面接完了時の最終合否判定をClient設定ベースで制御する仕組みを実装した。従来のハードコード閾値（70点固定）をSituation単位の設定に置き換え、柔軟なリジェクト条件を定義可能にした。

機能	説明
Client設定ベース閾値	Situation単位で合格基準・必須質問閾値・連続不合格上限を設定可能
リアルタイム判定	各回答評価後に即座にリジェクト判定。必須質問不合格 / 連続不合格で即終了
完了時最終判定	面接完了時に平均スコアで最終合否を判定
悲観的ロック	非同期評価ジョブとの競合を防止する排他制御

2. DB変更（マイグレーション）

ファイル: db/migrate/20260310120000_add_auto_reject_to_situations.rb

situations テーブル

カラム	型	デフォルト	説明
passing_score	integer	70	面接全体の合格基準（平均スコア）
auto_reject_enabled	boolean	true	自動リジェクト機能の有効/無効
reject_on_required_fail	boolean	true	必須質問不合格で即リジェクト
min_required_score	integer	70	個別回答の合格閾値
max_consecutive_fails	integer	0	連続不合格の上限（0=無効）
reject_notify_method	string	in_app	通知方法（in_app / email / none）

interviews テーブル

カラム	型	デフォルト	説明
rejection_reason	string	null	リジェクト理由
rejected_at	datetime	null	リジェクト日時

interview_results テーブル

カラム	型	デフォルト	説明
rejection_details	json	{}	リジェクト判定の詳細 (reason_code, score, threshold等)

後方互換性: 全カラムにデフォルト値が設定されており、既存データは従来と同じ70点基準で動作する。
max_consecutive_fails = 0 で連続不合格チェックは無効。

3. リジェクト判定サービス (RejectJudge)

ファイル: app/services/interview_engine/reject_judge.rb

設計方針

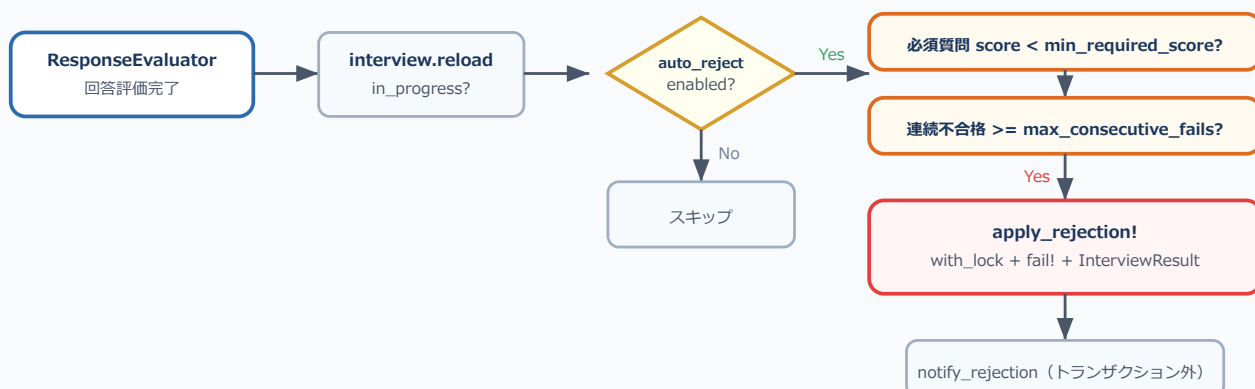
項目	方針
責務	リジェクト判定ロジックの一元管理 (判定 + 適用)
判定結果	RejectDecision Struct (rejected, reason, details) で構造化返却
排他制御	with_lock でinterviewの悲観的ロックを取得後にステータス再確認
閾値分離	min_required_score (個別回答) と passing_score (面接全体) を分離

リジェクト条件一覧

reason_code	判定タイミング	条件
required_question_failed	リアルタイム	必須質問のスコア < min_required_score
consecutive_fails	リアルタイム	連続不合格数 >= max_consecutive_fails
below_passing_score	面接完了時	平均スコア < passing_score

4. リアルタイム判定フロー

回答評価後のリジェクト判定フロー



悲観的ロックによる排他制御

```
def apply_rejection!(decision)
  return unless decision.rejected?

  @interview.with_lock do
    # ロック取得後にステータスを再確認（非同期ジョブとの競合防止）
    unless @interview.in_progress?
      Rails.logger.info("Skipping rejection: already #{@interview.status}")
      return
    end

    @interview.update!(rejection_reason: decision.reason, rejected_at: Time.current)
    @interview.fail!
    InterviewResult.create!(interview: @interview, final_status: :failed, ...)
  end
end
```

5. 完了時最終判定

SessionManager#complete_interview から RejectJudge#judge_on_completion を呼び出し、平均スコアが passing_score 未満の場合にリジェクトする。

```
def judge_on_completion(responses)
  return not_rejected unless @situation.auto_reject_enabled?

  scores = responses.map(&:score).compact
  average_score = (scores.sum.to_f / scores.count).round(2)

  if average_score < @situation.passing_score
    reject("below_passing_score", "平均スコアが合格基準未達", ...)
  else
    not_rejected
  end
end
```

6. 既存コード改修

ファイル	変更内容
response_evaluator.rb	ハードコード PASS_THRESHOLD → min_required_score に置換。 check_interview_continuation → check_rejection に変更（RejectJudge 委譲）。トランザクション外でリジェクト判定実行。
session_manager.rb	generate_interview_result に RejectJudge 統合。fail_interview に with_lock + 重複防止チェック追加。passing_score を結果データに含める。通知メソッド追加。
situation.rb	リジェクト設定のバリデーション追加（passing_score 0-100, reject_notify_method in_app/email/none）
interview.rb	rejected? メソッド追加
interview_result.rb	rejection_details JSONカラム直接アクセス、rejected? メソッド追加
interviews_controller.rb	status / complete レスポンスにリジェクト情報追加

7. コードレビュー結果

重要度	件数	主な内容
Critical	4	トランザクション不整合、レースコンディション、JSON型/store不整合、InterviewResult 重複作成
Warning	5	N+1（consecutive_fails）、passing_score二重利用、循環依存、レスポンス型不整合
Info	4	auto_reject_enabledデフォルト値、truncate_text重複、二重判定防止

Critical修正の詳細

#	問題	修正内容
1	apply_rejection! のトランザクション不整合	with_lock で fail! + InterviewResult.create! を一括実行。通知はロック外
2	非同期ジョブと complete の競合	apply_rejection! / complete_interview 双方で悲観的ロック取得。interview.reload でステータス再確認
3	rejection_details の store + JSON 型の二重エンコード	store 宣言を削除、JSONカラムとして直接アクセス
4	リジェクト後の complete で InterviewResult 重複作成	generate_interview_result / fail_interview 冒頭で既存チェック

8. 変更ファイル一覧

[db/migrate/20260310120000_add_auto_reject_to_situations.rb](#)

新規 リジェクト設定カラム
(situations/interviews/interview_results)

[app/services/interview_engine/reject_judge.rb](#)

新規 リジェクト判定サービス (リアルタイム + 完了時判定、悲観的ロック)

[app/services/interview_engine/response_evaluator.rb](#)

改修 ハードコード閾値撤廃、RejectJudge統合、トランザクション外判定

[app/services/interview_engine/session_manager.rb](#)

改修 RejectJudge統合、with_lock、重複防止、通知メソッド

[app/models/situation.rb](#)

改修 リジェクト設定バリデーション、predicate メソッド

[app/models/interview.rb](#)

改修 rejected? メソッド追加

[app/models/interview_result.rb](#)

改修 rejection_details直接アクセス、rejected? メソッド

[app/controllers/api/interviews_controller.rb](#)

改修 status/completeにリジェクト情報追加

Day 8 — 音声AI面接の基盤実装

9. 概要

既存のSTT/TTS/MediaProcessorを強化し、音声面接のEnd-to-Endフローを統合する
AudioInterviewService を新規作成した。タイムアウト・リトライ・バリデーション・セキュリティを全面的に強化した。

コンポーネント	API/ツール	強化内容
STTClient（音声認識）	OpenAI Whisper	リトライ、ファイルバリデーション、タイムアウト、FDリーク修正
TTSClient（音声合成）	OpenAI TTS	リトライ、テキスト長制限、ボイスマップ、タイムアウト
MediaProcessor	ffmpeg	ffmpegチェック、タイムアウト、音声長バリデーション、正規化
AudioInterviewService	--	音声フロー統合、一時ファイル管理、事前生成バッチ

10. STTClient 強化（音声認識）

ファイル: app/services/interview_engine/stt_client.rb

追加機能

機能	設定	説明
ファイルバリデーション	MAX_FILE_SIZE: 25MB	Whisper API上限に準拠。サイズ・形式・空ファイルチェック
対応形式	.mp3, .mp4, .mpeg, .mpga, .m4a, .wav, .webm	Whisper APIの受付形式に限定
リトライ	MAX_RETRIES: 2, DELAY: 指数	429/5xx/タイムアウトでリトライ
タイムアウト	60秒（接続/読取り）	長時間音声のSTT処理に対応
FDリーク修正	--	File.open ブロック形式でファイルハンドルを確実にクローズ

エラー分類

エラー	HTTPコード	リトライ
STTTimeoutError	429, 500-599, timeout	する（最大2回）
STTError	400-428, 430-499	しない

11. TTSClient 強化（音声合成）

ファイル: app/services/interview_engine/tts_client.rb

追加機能

機能	設定	説明
テキスト長制限	MAX_TEXT_LENGTH: 4096	OpenAI TTS入力上限に準拠
ボイスマップ	ja: nova, en: nova	言語ごとの推奨ボイス（拡張可能）
リトライ	MAX_RETRIES: 2, DELAY: 指数	429/5xx/タイムアウトでリトライ
タイムアウト	30秒（接続/読取り）	
ファイル名衝突回避	SecureRandom.hex(8)	タイムスタンプ方式から変更

12. MediaProcessor 強化

ファイル: app/services/interview_engine/media_processor.rb

追加メソッド

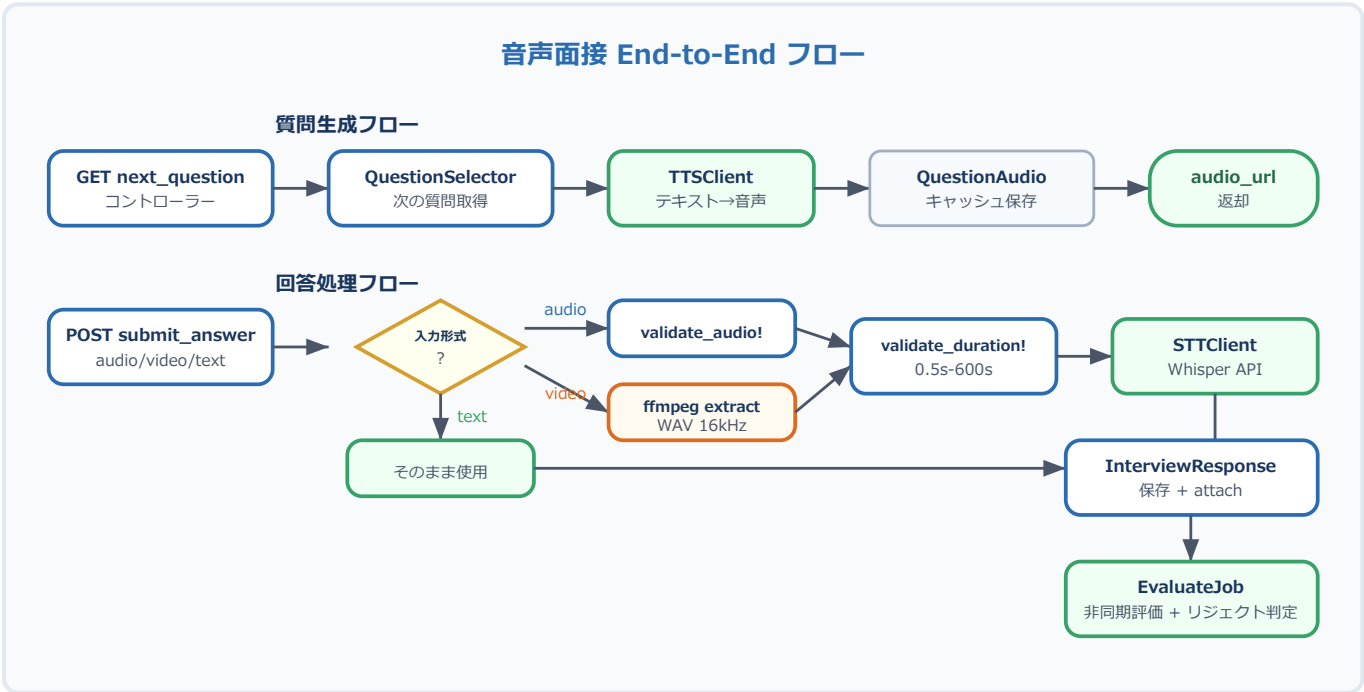
メソッド	説明
audio_duration(path)	ffprobeで音声の長さ（秒）を取得
validate_audio_duration!(path)	音声長のバリデーション（0.5秒-600秒）
normalize_audio(path)	音声をWAV 16kHz monoに正規化（STT最適化）

セキュリティ強化

対策	内容
ffmpegチェック	ensure_ffmpeg! でインストール確認 (false時は毎回再チェック)
ファイルサイズ	MAX_VIDEO_SIZE: 500MB
タイムアウト	FFMPEG_TIMEOUT: 120秒 (Open3.capture3 + Timeout)
private化	private_class_method で内部メソッドを隠蔽

13. AudioInterviewService (統合サービス)

ファイル: app/services/interview_engine/audio_interview_service.rb



主要メソッド

メソッド	説明
prepare_question(question)	質問の音声データ取得 (TTS + キャッシュ)
process_answer_media(...)	音声/動画/テキストから回答テキストを取得
cleanup_temp_files(*paths)	一時ファイルのクリーンアップ
self.pregenerate_question_audio(situation)	全質問の音声を事前生成 (バッチ用、N+1回避済み)

14. コントローラー統合

submit_answer アクションを AudioInterviewService に統合し、以下を改善:

改善項目	内容
二重回答防止	with_lock でinterviewロック後に重複チェック。ロック外で音声処理
一時ファイル管理	ensure ブロックで例外時もクリーンアップ
エラーハンドリング	AudioError / STTError / MediaError を個別に rescue
認可強化	secure_compare でトークンのタイミングセーフ比較

15. 運用ツール (Rakeタスク)

ファイル: lib/tasks/audio.rake

タスク	説明
rake audio:pregenerate[SITUATION_ID]	指定Situationの全質問の音声を事前生成
rake audio:pregenerate_all	全アクティブSituationの音声を事前生成
rake audio:cleanup	tmp/interview_audio/ の24時間以上前のMP3を削除
rake audio:cleanup_tmpdir	Dir.tmpdir の一時WAV/MP3を削除

16. コードレビュー結果

重要度	件数	主な内容
Critical	4	FDリーク、認可ロジック不備、二重回答でSTT二重課金、ffmpegキャッシュ問題
Warning	7	429エラー種別、ディスク容量、cleanup漏れ、N+1、タイミングセーフ比較
Info	5	リトライ重複、private_class_method、cleanup範囲、フロー統一

Critical修正の詳細

#	問題	修正内容
1	File.new でファイルハンドルがリーク	File.open ブロック形式に変更（自動クローズ）
2	authorize_interview! トークン不一致時のフォールスルー	トークン提供時は一致/不一致で完結。 secure_compare 使用
3	二重回答送信でSTT二重課金	with_lock で重複チェック後にロック外で音声処理
4	ffmpegチェックのfalseキャッシュ問題	false 時は毎回再チェックするよう修正

17. 変更ファイル一覧

app/services/interview_engine/stt_client.rb

改修 リトライ、バリデーション、タイムアウト、FDリーク修正

app/services/interview_engine/tts_client.rb

改修 リトライ、テキスト長制限、ボイスマップ、タイムアウト

app/services/interview_engine/media_processor.rb

改修 ffmpegチェック、タイムアウト、音声長バリデーション、正規化

app/services/interview_engine/audio_interview_service.rb

新規 音声面接フロー統合（STT/TTS/Media + 事前生成バッチ）

app/controllers/api/interviews_controller.rb

改修 AudioInterviewService統合、with_lock、ensure cleanup、secure_compare

lib/tasks/audio.rake

新規 音声事前生成 / クリーンアップタスク

付録: 全APIエンドポイント一覧 (Day 8時点)

メソッド	パス	認証	概要
POST	/api/interviews/start	Devise/Token	面接開始 (自動復帰対応)
POST	/api/interviews/start_by_token	Token のみ	URL即時開始/自動復帰
GET	/api/interviews/:id/next_question	Devise/Token	次の質問取得 + TTS音声URL (タイムアウトチェック付き)
POST	/api/interviews/:id/submit_answer	Devise/Token	回答送信 (音声/動画/テキスト対応、STT自動変換、タイムアウトチェック付き)
POST	/api/interviews/:id/complete	Devise/Token	面接完了 (リジェクト判定 + 結果生成)
GET	/api/interviews/:id/status	Devise/Token	状態取得 (rejected, rejection_reason含む)
POST	/api/interviews/:id/resume	Devise/Token	中断面接の再開

フェーズ2 (Day 4-8) 完了: 質問分岐エンジン、LLM制御レイヤー、セッション管理、自動リジェクト判定、音声AI面接基盤の全機能が実装完了。フェーズ3 (Day 9-13: API設計・Rails連携・データ保存) に進む準備が整った。